

DD_FA-03 - Customer KYC Field Audit - Developer Implementation Guide

Version: v1.0 **Bundle:** 10_other **Companion DD:** DD_FA-03-Customer_KYC_Field_Audit-v1.0

1. Goal

Implement field verification for customer KYC, address, contact, consent, and duplicate-risk cases.

FA-03 records audit evidence and discrepancy routing. ILM-CFG-01 remains the customer/KYC source of truth.

2. Runtime Flow

```
sequenceDiagram
    participant BO as Backoffice
    participant FA as FA-03
    participant ILM as ILM-CFG-01
    participant WO as WO-01
    participant MOB as Mobile App
    participant APR as EM-CFG-04
    participant TCK as TCK-01
    participant INT as CUST-INT

    BO->>FA: POST /api/kyc-field-audits/tasks
    FA->>ILM: read masked customer/KYC snapshot
    FA->>FA: store expected snapshot
    FA->>WO: optional FIELD_KYC_AUDIT WO
    MOB->>FA: submit observations/evidence
    FA->>FA: compare expected vs observed
    FA->>FA: run Drools routing
    FA->>APR: approval for correction/override
    FA->>ILM: request approved correction
    FA->>TCK: create follow-up case when needed
    FA->>INT: record interaction
```

3. Step 1 - Migrations

Create:

- kyc_field_audit_campaign
- kyc_field_audit_task
- kyc_field_audit_expected_snapshot
- kyc_field_audit_observation
- kyc_field_audit_discrepancy

Security requirements:

- Do not store raw ID numbers.
- Store only masked KYC values in snapshots.
- Store file ids, not binary files.

4. Step 2 - Create Task

Endpoint:

```
POST /api/kyc-field-audits/tasks
Idempotency-Key: kfat-CUS-100045-202608
Content-Type: application/json
```

Request:

```
{
```

```

    "operatorCode": "WIK",
    "taskType": "ADDRESS_VERIFY",
    "customerId": "CUS-100045",
    "accountId": "ACC-900045",
    "orderId": "ORD-1001",
    "homepassId": "HP-GPON-0440",
    "assignedToUserId": "usr-tech-030",
    "createWorkOrder": true
  }
}

```

FA-03 behavior:

1. Validate permission through EM-CFG-03.
2. Apply idempotency.
3. Insert task in CREATED.
4. Call ILM:

```
GET /api/customers/CUS-100045/kyc-summary?masked=true
```

5. Insert kyc_field_audit_expected_snapshot.
6. Create WO if requested.
7. Emit KycFieldAuditTaskCreated.

5. Step 3 - Create WO

```

POST /api/work-orders
Idempotency-Key: kfat-001-wo
Content-Type: application/json

{
  "operatorCode": "WIK",
  "kind": "FIELD_KYC_AUDIT",
  "originatingContextType": "KYC_FIELD_AUDIT_TASK",
  "originatingContextId": "kfate-001",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "homepassId": "HP-GPON-0440",
  "metadata": {
    "requiredEvidence": ["gps", "customerConfirmation", "addressEvidence", "consent"]
  }
}

```

6. Step 4 - Mobile Observation

```

POST /api/kyc-field-audits/tasks/kfat-001/observations
Idempotency-Key: mob-kyc-888
Content-Type: application/json

{
  "observationType": "ADDRESS_CONFIRMED",
  "resultStatus": "PASS",
  "observedValue": {
    "gpsMatch": true,
    "occupantConfirmed": true
  },
  "evidenceFileIds": ["file-kyc-001"],
  "gpsLatitude": -1.2921,
  "gpsLongitude": 36.8219,
  "capturedAt": "2026-08-22T09:30:00Z",
  "offlineClientRef": "mob-kyc-888"
}

```

Validate:

- field user assignment;
- file ids and restricted file category;
- no raw ID number is present in observedValue;
- duplicate offline ref returns existing observation.

7. Step 5 - Submit And Compare

On submit:

1. Load masked ILM snapshot.
2. Load observations.
3. Create discrepancies for IDENTITY_MISMATCH, ADDRESS_MISMATCH, CONTACT_MISMATCH, MISSING_CONSENT, DUPLICATE_SUSPECTED, or CUSTOMER_REFUSED.
4. Run Drools for severity/routing.
5. Close if clean.

8. Step 6 - Drools

```
public record KycFieldAuditFact(  
    String operatorCode,  
    String discrepancyType,  
    boolean evidenceComplete,  
    boolean customerConfirmed,  
    boolean fraudSuspected,  
    boolean orderBlocked  
) {}  
  
package rules.field_audit.kyc.severity  
  
rule "Identity mismatch critical"  
when  
    $f : KycFieldAuditFact(discrepancyType == "IDENTITY_MISMATCH")  
    $d : KycFieldAuditDecision()  
then  
    $d.setSeverity("CRITICAL");  
    $d.route("REQUEST_KYC_REVIEW");  
end  
  
package rules.field_audit.kyc.routing  
  
rule "Address correction needs approval"  
when  
    $f : KycFieldAuditFact(discrepancyType == "ADDRESS_MISMATCH",  
                           evidenceComplete == true)  
    $d : KycFieldAuditDecision()  
then  
    $d.route("REQUEST_ILM_CORRECTION");  
    $d.requireApproval("KYC_FIELD_AUDIT_ADDRESS_CORRECTION");  
end
```

9. Step 7 - Route

For ILM correction:

```
POST /api/customer-corrections  
Idempotency-Key: kfad-001-ilm-correction  
Content-Type: application/json  
  
{  
    "operatorCode": "WIK",  
    "originRefType": "KYC_FIELD_AUDIT_DISCREPANCY",  
    "originRefId": "kfad-001",  
    "customerId": "CUS-100045",  
    "correctionType": "ADDRESS_CORRECTION",  
    "evidenceFileIds": ["file-kyc-001"]  
}
```

For customer follow-up:

```
POST /api/tickets  
Idempotency-Key: kfad-001-ticket  
Content-Type: application/json  
  
{  
    "operatorCode": "WIK",  
    "ticketType": "KYC_FIELD_AUDIT_FOLLOW_UP",  
    "customerId": "CUS-100045",  
    "originRefType": "KYC_FIELD_AUDIT_DISCREPANCY",
```

```
    "originRefId": "kfad-001"  
  }
```

Always write a customer interaction record after contact or outcome.

10. Tests

1. Snapshot masks sensitive fields.
2. Raw ID values are rejected.
3. Duplicate mobile submit is idempotent.
4. Address mismatch creates discrepancy.
5. Identity mismatch is critical.
6. ILM correction is called only after required approval.
7. Ticket creation uses TCK API.
8. Interaction is written to CUST-INT.